

Why Can Large Language Models Talk and Think Like Humans?

Hang Li, Shaohua Zhang, Yuan Lin

1 Introduction

We use Large Language Models (LLMs) every day. One obvious feeling is that they seem to truly understand our language, although they sometimes hallucinate. On the other hand, observing the chain of thought output by LLMs, which is the linguistic representation of their reasoning process, we feel that they can indeed think like humans.

This article discusses: **What are the language and thinking capabilities of LLMs? How are these capabilities formed through their implementation principles, methods, and working mechanisms?** (see Figure 1)

LLM technology is created by humans, and its implementation principles are clear, but its working mechanisms (Mechanics) are still not fully understood. The enormous scale and extreme complexity of LLMs present great difficulties for research on their capabilities.

Since the advent of ChatGPT, there has been extensive research on LLM mechanisms and characteristics, particularly recent research on working mechanisms (or interpretability). These works have provided answers to varying degrees on this core AI topic from different perspectives. However, many questions remain for future research.

This article will summarize the basic principles and implementation methods of LLMs, and provide a brief introduction to research on LLM working mechanisms, including ByteDance’s work on LLM memory mechanisms. Based on this, we will present our views on how LLM capabilities are formed. This article contains three main arguments.

- **LLMs learn patterns of language use and reasoning, importantly including higher-order patterns.** LLM learning belongs to machine learning, and what is learned is essentially statistical regularities in the data, or patterns in the data. Language data is rich in content, containing vocabulary, grammar, semantics, pragmatic information, and world knowledge. We can see that LLMs not only learn lower-order patterns related to vocabulary and grammar, but also higher-order patterns related to semantics, pragmatics, and world knowledge. Previous language models often could not achieve this, which is precisely the “emergent” capability

of ChatGPT and subsequent LLMs. Therefore, the view that LLMs only learn the form of language without learning content (such as Chomsky’s view mentioned later) is not convincing.

- **The basic implementation principle can be summarized as Next Token Prediction (NTP), but the overall capability is jointly determined by strategy, model, algorithm, and data.** The learning and inference process of LLMs is NTP, but this is only the surface form; the specific implementation methods and their characteristics are more important. The maximum likelihood estimation used in pre-training (equivalent to data compression) estimates the probability distribution of token sequence data. Reinforcement learning in post-training aims to fine-tune the model to become the optimal policy function for token sequence generation. Transformer as a model has extremely strong language and knowledge representation capabilities. The stochastic gradient descent optimization algorithm helps find solutions with good generalization. The key to LLMs lies in the systematic integration and large-scale implementation of these technologies. The view that simply attributes LLM success to NTP is an oversimplification.
- **The internal mechanisms of LLMs have been analyzed and understood to some extent.** In recent years, LLM interpretability research has made certain progress, and LLMs are no longer completely black boxes to us. Features in LLMs can be extracted using tools such as SAE, and circuits formed between features can be tracked using tools such as CLT. ByteDance’s recent work further reveals the patterns by which features in LLMs are memorized during learning and retrieved during reasoning. As future research continues to deepen, the working mechanisms of LLMs will be increasingly analyzed and understood by us.



Figure 1: The relationship between LLM’s language and thinking capabilities, working mechanisms, and implementation principles and methods.

2 Human Language Understanding and Thinking

First, we organize the characteristics of human language understanding and thinking.

2.1 Language Understanding

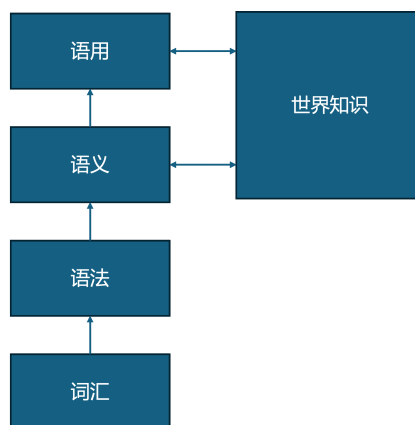


Figure 2: Human language understanding process

Language processing is divided into understanding and generation. Human language understanding can be defined from internal and external perspectives. The internal perspective refers to understanding of language in the human mind; the external perspective is observed human understanding of language.

Internal language understanding involves the representation of language in the human mind. This includes concept grounding, that is, connecting heard or seen language with existing knowledge. For example, when a person hears "giant panda," they connect their knowledge about giant pandas.

External language understanding can often be viewed from the perspective of task completion, with goals and results. Dialogue, such as question answering, can also be seen as a process of completing tasks. (The Turing test judges whether a computer can understand language from an external perspective.)

The embodied cognition hypothesis believes that human language understanding is achieved through the generation of imagery in consciousness. For example, when answering the question "Does a gorilla have a nose?", an image of a gorilla emerges in the mind.

Language is sequential data in form, whether speech or text in images, and usually needs to be processed in order; but in the human brain, its understanding process involves hierarchical and parallel processing.

Language simultaneously contains vocabulary, grammar, semantics, and pragmatic information, while semantics and pragmatics are closely related to world knowledge (see Figure 2). From the perspective of abstraction level, the former two belong to low-level information, while the latter two belong to high-level information. In the above example, "gorilla," "have," and "nose" belong to semantic information, while the intent expressed by the entire interrogative sentence belongs to pragmatic information.



Figure 3: Chunk analysis example: phrases in boxes are chunks

Regardless of language type or grammatical differences, languages can usually be viewed as composed of chunks of different granularities and can be analyzed from the perspective of chunks (see Figure 3).

2.2 Thinking

During the thinking process, neurons in the human brain's nervous system are activated, forming neural representations in the brain. At the same time, in the human mind, especially at the conscious level, symbolic representations or imagery are formed. The generation of symbolic representations is an important view of traditional cognitive science, while the generation of imagery is advocated by embodied cognition.

Embodied cognition believes that human reasoning largely depends on analogy. For example, thinking "this phone is the same brand as the one I used before, so the quality should also be good."

In rigorous mathematical derivation, logical reasoning rules (such as syllogism) or computational rules (such as arithmetic operations) are mainly used. But these are only present in specific scenarios.

Human reasoning processes can be characterized using Bayesian inference. For example, Sherlock Holmes' detective reasoning can be seen as Bayesian inference.

3 LLM Learning and Reasoning

LLM is a language model, and its processing form can be summarized as Next Token Prediction (NTP), but its overall mechanism is more complex and cannot be simply understood with this concept.

In the pre-training stage, maximum likelihood estimation is performed on token sequence data to obtain the optimal model for generating token sequences. In the post-training stage, reinforcement learning is used to further fine-tune the model to make it the optimal policy in the sense of generating reasonable token sequences; the implementation learns a **token-level** policy, but can be approximately regarded as learning a **sequence-level** policy.

In terms of model architecture, LLMs adopt autoregressive Transformer decoders; in terms of optimization algorithms, stochastic gradient descent (such as AdamW) is used.

The language capabilities of LLMs are the result of the combined effects of model structure, optimization algorithm, training strategy, and language data characteristics. Pre-training determines basic language and reasoning capabilities, while post-training further enhances reasoning capabilities and task completion capabilities.

This section summarizes the basic principles and implementation methods of LLMs. The content involves technical details, which readers can skip according to their circumstances.

3.1 Pre-training

In pre-training, LLMs represent the generation probability of token sequences. According to the probability multiplication theorem, the probability of token sequence $\mathbf{w}$ can be written as

$$P_{\boldsymbol{\theta}}(\mathbf{w}) = \prod_{t=1}^T P_{\boldsymbol{\theta}}(w_t | w_{<t}), \quad \mathbf{w} = w_1 w_2, \dots, w_T \quad (1)$$

The learning objective is to estimate model parameters $\boldsymbol{\theta}$ to compute conditional probabilities $P_{\boldsymbol{\theta}}(w_t | w_{<t})$. The prediction objective is to autoregressively generate token sequences based on conditional probabilities $P_{\boldsymbol{\theta}}(w_t | w_{<t})$. LLMs typically use Transformer decoders as models to represent conditional probabilities $P_{\boldsymbol{\theta}}(w_t | w_{<t})$.

Learning is performed through maximum likelihood estimation, that is, maximizing the likelihood function of training data, or equivalently minimizing cross-entropy loss. Estimating model parameters is equivalent to **modeling** in lossless compression of training data. The cross-entropy loss is written as

$$L(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{w} \sim P_{\text{data}}} \left[\sum_{t=1}^T -\log P_{\boldsymbol{\theta}}(w_t | w_{<t}) \right] \quad (2)$$

Its gradient function is

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{w} \sim P_{\text{data}}} \left[\sum_{t=1}^T -\nabla_{\boldsymbol{\theta}} \log P_{\boldsymbol{\theta}}(w_t \mid w_{<t}) \right] \quad (3)$$

Lossless compression first estimates a model of the data distribution, and second encodes the data based on the model. Here there is only the first step of modeling, and not the second step of encoding. Ilya Sutskever’s view that ”compression is intelligence” can be interpreted as an interpretation of LLM pre-training modeling.

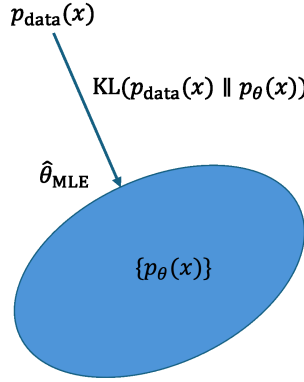


Figure 4: The distribution obtained by maximum likelihood estimation converges in probability to the KL projection of the true distribution onto the parametric distribution family.

Maximum likelihood estimation has excellent properties under asymptotic conditions, that is, when the amount of data tends to infinity. Consider the probability distribution space, where the learning objective is to find the optimal distribution in the parametric distribution family $\{p_{\boldsymbol{\theta}}(\mathbf{x})\}$. The parametric distribution family forms a manifold in space. Assume that the true data distribution $p_{\text{data}}(\mathbf{x})$ is located outside the manifold, as shown in Figure 4. Under certain regularity conditions, the distribution represented by the maximum likelihood estimation parameter $\hat{\boldsymbol{\theta}}_{\text{MLE}}$ converges in probability to the KL projection of the true distribution onto the manifold, that is, the distribution with the smallest difference from the true distribution in the sense of KL divergence.

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} = \arg \min_{\boldsymbol{\theta} \in \Theta} \text{KL}(p_{\text{data}}(\mathbf{x}) \parallel p_{\boldsymbol{\theta}}(\mathbf{x})) \quad (4)$$

If the true distribution exists within the parametric distribution family, then the distribution represented by the maximum likelihood estimation parameter $\hat{\boldsymbol{\theta}}_{\text{MLE}}$ converges in probability to the true distribution.

3.2 Post-training

LLM post-training includes two stages: SFT (Supervised Fine-Tuning) and RL (Reinforcement Learning). The latter’s special case is RLHF (Reinforcement Learning from Human Feedback).

In RL for post-training, the LLM is a policy function, where the state is the token sequence so far, and the action is the next token. The reward is generally defined at the last position of the token sequence, representing the rationality of the sequence, with rewards at other positions being zero.

Post-training RL typically adopts policy gradient methods, using the policy function to perform rollout of action trajectories, that is, token sequences. SFT can be seen as imitation learning, with the goal of fitting given example data.

The learning objective is to maximize return, that is, the expectation of reward, to fine-tune the model parameters θ of the LLM. The policy function $\pi_\theta(y_t|\mathbf{x}, y_{<t})$ represents the generation probability of the next token y_t given the prompt $\mathbf{x}$ and the generated prefix $y_{<t}$. In the simplest REINFORCE algorithm, the gradient function of return is written as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}, \mathbf{y} \sim \pi_\theta} \left[R(\mathbf{x}, \mathbf{y}) \sum_{t=1}^T \nabla_\theta \log \pi_\theta(y_t | \mathbf{x}, y_{<t}) \right] \quad (5)$$

where $\mathbf{x}$ represents the token sequence of the prompt, $\mathbf{y}$ represents the token sequence of the response, and $R(\mathbf{x}, \mathbf{y})$ represents its reward. In actual methods (such as PPO, GRPO), advantage functions and regularization terms are introduced to stabilize training, but they all belong to policy gradient methods.

From the form of the generation process (Equations (3) and (5)), the post-training stage still generates tokens one by one, which has similarity with pre-training. The difference is that the goal of post-training is no longer to learn the data distribution, but to learn to generate sequences with maximum expected (average) reward.

In fact, post-training RL can be defined at two levels: **token level** and **sequence level**. Current RL implementations are all at the token level, as described above. But from the perspective of conceptual understanding, sequence-level RL is more intuitive. Token-level RL is usually an approximation of sequence-level RL [9].

The goal of sequence-level RL is to learn how to generate responses $\mathbf{y}$ with maximum expected reward given prompt $\mathbf{x}$. Its gradient can be expressed as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}, \mathbf{y} \sim \pi_\theta} \left[R(\mathbf{x}, \mathbf{y}) \nabla_\theta \log \pi_\theta(\mathbf{y}|\mathbf{x}) \right] \quad (6)$$

where the reward function $R(\mathbf{x}, \mathbf{y})$ evaluates the rationality of the response, and the policy π_θ generates complete sequences autoregressively token by token. In this simple case, token-level and sequence-level RL are equivalent.

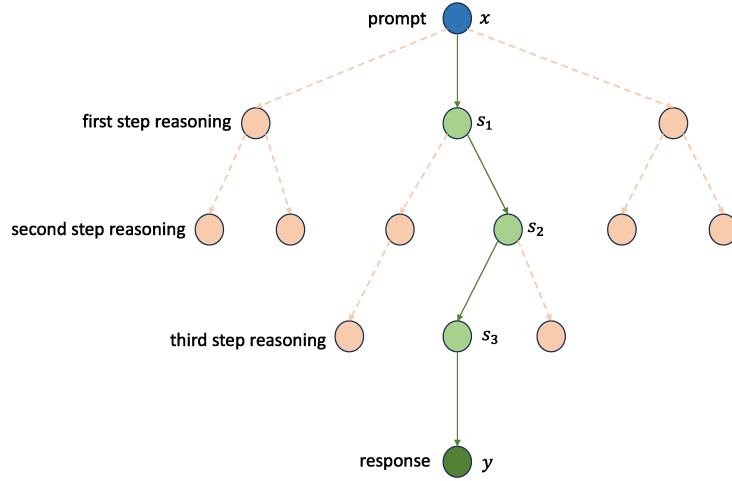


Figure 5: Illustration of LLM reasoning as search in statement space

3.3 Reasoning

It can be considered that LLM reasoning is a search in the huge space composed of token sequences. We call these token sequences "statements". Statements express certain content and are the basic unit of reasoning. They can be natural language (such as a piece of text) or formal language (such as a piece of code); they can express rigorous logical reasoning or analogical reasoning; a chain of thought may contain multiple statements.

During the search process, LLMs continuously select appropriate reasoning paths and finally arrive at answers (see Figure 5). After RL training, model parameters remain fixed during the reasoning stage, but more sufficient searches can be performed in the statement space by generating longer chains of thought, sampling more paths, etc., to find better answers. This process can achieve test-time scaling.

RL training can enhance the reasoning capabilities of models. By optimizing policies, it makes models more inclined to generate high-quality intermediate and final results, essentially enhancing search capabilities [6].

Formalizing reasoning as search in statement space enables LLMs to transcend pure language understanding and generation, moving toward higher-level intelligence with "thinking" capabilities.

3.4 Optimization Algorithm: Stochastic Gradient Descent

After the learning strategy is determined, the next step is to use optimization algorithms to estimate optimal parameters to optimize the objective function. As long as the loss function of a neural network is differentiable with respect to parameters, it can be optimized through stochastic gradient descent (SGD)

and its variants. The same applies to LLMs.

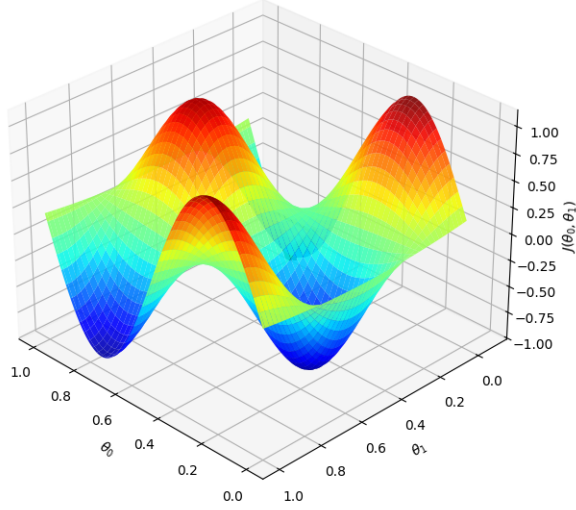


Figure 6: Stochastic gradient descent

Experiments show that using SGD and its variants (such as AdamW) not only allows the loss function to converge to a good local minimum region, but also the learned model usually has good generalization ability. (See Figure 6)

SGD tends to converge to flatter local minimum regions, which plays an important role in the generalization performance of deep learning. Moreover, SGD usually has better generalization effects when the batch processing scale is smaller [5].

Experiments show that deep learning models are often over-parameterized, that is, model capacity far exceeds data scale. This is different from traditional machine learning. The training loss function exhibits a double descent phenomenon [1] (Figure 7 is a schematic). In the under-parameterized region, as model parameters increase, training loss decreases while test loss increases; traditional machine learning is in this region. After the training loss approaches zero (past the interpolation threshold, i.e., the critical point where the model completely fits the training data), entering the over-parameterized region, as model capacity continues to increase, training loss continues to approach zero, and test loss also decreases; deep learning is in this region [7].

Experience shows that LLM pre-training is also in the over-parameterized region, so good generalization effects can be achieved through SGD, and test loss follows scaling law [4]. Scaling law refers to the pattern that loss function decreases with power law with respect to computation, parameter amount, and data amount. Similar trends are also observed in post-training (RL), but the mechanism is different and requires further research.

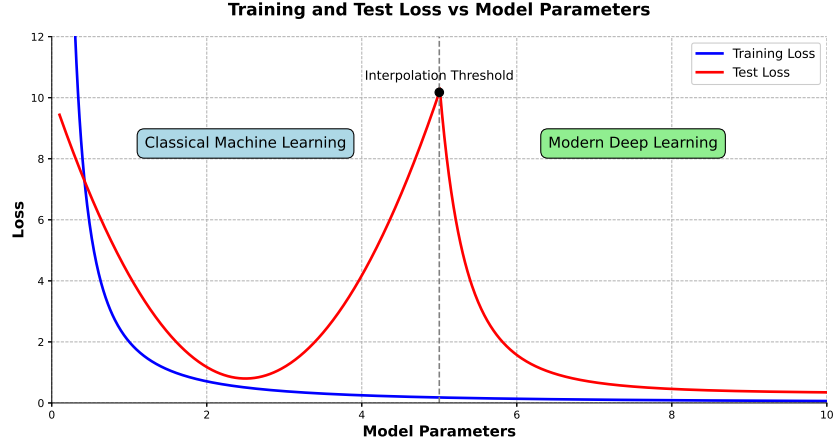


Figure 7: Training and prediction loss curves

3.5 Model: Transformer

LLMs typically use Transformer decoder architecture. It plays an important role in language and knowledge representation. Especially self-attention, feed-forward networks (FFN), and residual connections in Transformers.

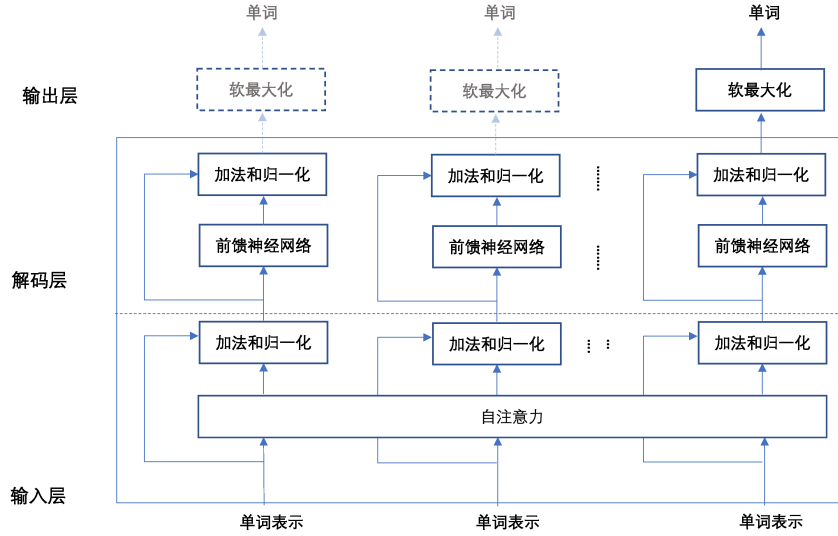


Figure 8: Transformer decoder architecture

Figure 8 shows the architecture diagram of the Transformer decoder. Table 1 lists the model scales of representative LLMs.

Model	Training Data	Parameters	Layers	Context Length	Vocab Size	Model Dim
GPT-1	1B Tokens	117M	12	512	40k	768
GPT-2	10B Tokens	1.5B	48	1024	~50k	1600
GPT-3	300B Tokens	175B	96	2048	~50k	12288

Table 1: GPT model scale, training data, context window, and vocabulary size

Self-Attention

Each layer in the Transformer has a self-attention module. The main role of self-attention is to linearly combine input representation vectors with context representation vectors to produce new output representation vectors.

First, linear transformations are applied to the input representation vector at the current position and its contextual input vectors to obtain query, key, and value matrices. Attention is computed as follows

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V \quad (7)$$

where Q (Query), K (Key), and V (Value) are the query, key, and value matrices obtained by linear transformation of input tokens, d is the dimension of key vectors, and $\sqrt{d}$ is the scaling factor. Finally, the result is linearly transformed to obtain the output vector at the current position. To ensure that the vector at the current position is only linearly combined with vectors at previous positions, masking is required in matrix computation.

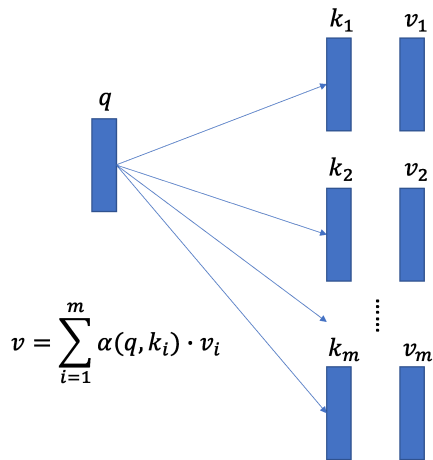


Figure 9: Illustration of self-attention computation

Intuitively, self-attention can be seen as a type of key-value store query:

for each query vector, compute its similarity with all key vectors, and use the similarity as weights to calculate the weighted sum of all value vectors as the result. The more the query matches the key, the greater the proportion of the corresponding value in the final result. It is a "soft" key-value store query. In extreme cases, if vectors are all one-hot vectors, self-attention computation is equivalent to symbol-based key-value store query. Figure 9 shows an example of self-attention computation.

The above describes single-head self-attention. In practice, multi-head self-attention is used, computing attention in parallel in different subspaces, then concatenating and linearly transforming. Different attention heads can focus on different types of compositional relationships, for example, some heads may focus on syntax, some on semantics, and some on pragmatics. Through the multi-head mechanism, the model can capture compositional relationships between representation vectors from multiple perspectives.

Self-attention is the composition of representation vectors. Related knowledge is also stored in the matrices of linear transformations. One of the most essential characteristics of language is compositionality: that is, words compose into phrases, phrases compose into sentences, and sentences compose into discourse. The self-attention mechanism exactly supports this compositionality. Each layer's attention computation recombines the input sequence, and multi-layer accumulated computation achieves progressive composition from local to global, from shallow to deep.

Feedforward Neural Network (FFN)

Each layer in the Transformer has a two-layer feedforward neural network (FFN) module.

$$\mathbf{y} = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \quad (8)$$

where $\mathbf{x}$ is the input vector, $\mathbf{W}_1$ and $\mathbf{W}_2$ are weight matrices, $\mathbf{b}_1$ and $\mathbf{b}_2$ are biases, ReLU is the activation function, and $\mathbf{y}$ is the output vector. Figure 10 shows a simple example of a two-layer FFN.

The main role of FFN is to perform feature detection on input vectors and represent detected features as output vectors. Related knowledge is also stored in the weight matrices of the neural network.

Each neuron in the intermediate layer of FFN can be seen as detecting features in the input vector. Specifically, the input vector $\mathbf{x}$ performs dot product operations with weight vectors (row vectors in $\mathbf{W}_1$). If the input direction is the same as the weight direction, the dot product result is positive, and the represented feature is activated after ReLU; if the directions are opposite, the dot product result is negative, and the represented feature is not activated after ReLU, thus obtaining the intermediate vector. Neurons in the output layer are linear neurons that convert intermediate vectors into output vectors through linear transformation (using $\mathbf{W}_2$). The dimensions of input and output vectors are the same, while the dimension of intermediate vectors (i.e., the number of neurons in the intermediate layer) is generally about 4 times larger, allowing more feature detection.

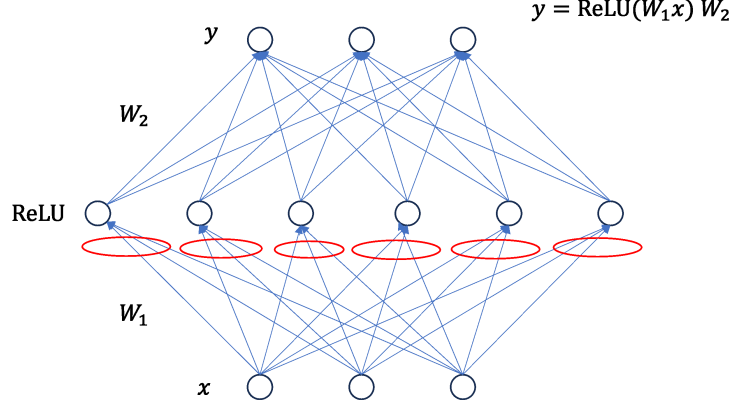


Figure 10: Illustration of two-layer feedforward neural network structure

FFN serves as a feature detector, and the training process enables the weight matrices to learn effective feature detection patterns. Corresponding knowledge is stored in weight matrices. In recent years, interpretability research has shown that neurons' representation of features is not "one-to-one" but exhibits superposition phenomena. This point will be detailed in Section 4.2.

The knowledge stored in FFN layers at different levels of Transformers varies. Research has found: shallow layers focus more on representing vocabulary and grammatical information; middle layers capture more semantic information; deep layers master information possibly related to context, pragmatics, and reasoning.

Modern LLMs are large-scale (see Table 1), and the expansion of parameter amounts enables them to accommodate large amounts of knowledge. Especially after introducing Mixture of Experts (MoE) structures in FFN layers, model capacity can be greatly improved, enabling them to represent and store richer knowledge.

Residual Connection

Transformers have residual connections after each layer's self-attention module and FFN module.

$$x_{l+1} = x_l + f(x_l) \quad (9)$$

where f represents self-attention or FFN. Based on LLM residual structures, representations learned at different layers can be accumulated and reused in deeper layers through additive paths.

LLMs can be understood as implicit ensembles of sub-networks of different depths formed by self-attention and FFN. As shown in Figure 11, assuming three functions are connected in series, if residual connections are added between them, the entire network can be interpreted as an implicit ensemble composed of sub-networks of different depths.

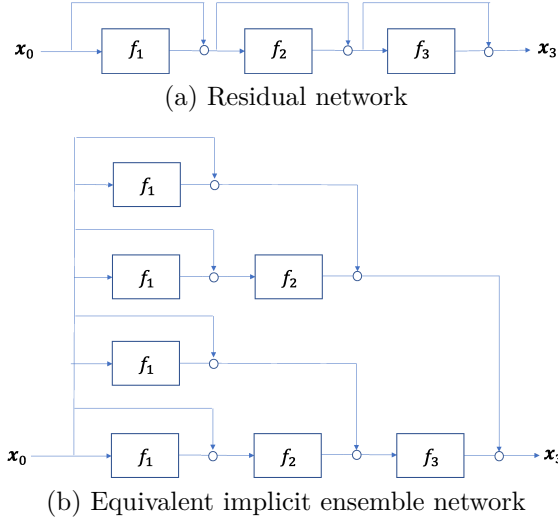


Figure 11: Example of residual neural network

Residual connections have the following roles: promoting cross-layer propagation of information and gradients, enabling shallow representations to directly affect deep layers, and improving the trainability of deep networks.

Residual connections bring more difficulties to LLM interpretability analysis. The attention weights of LLMs can be used to analyze correlations between tokens at different positions, but because information can propagate across layers through residual connections, these correlations do not necessarily come only from the current layer.

Other Mechanisms

LLMs explicitly represent the sequential positions of tokens through positional encoding (such as RoPE), which is significantly different from RNNs (Recurrent Neural Networks) that implicitly represent positional information through model structure. Layer normalization (usually in Pre-Norm form) is one of the important mechanisms for promoting stable Transformer training.

In addition, while maintaining the basic Transformer architecture unchanged, current LLMs have introduced multiple key improvement technologies. For example: Mixture of Experts (MoE) improves model capacity and computational efficiency; Swish activation function replaces ReLU activation function; SwiGLU structure improves FFN modules; Multi-Query Attention (MQA) or Grouped-Query Attention (GQA) improves attention computation efficiency. These improvements mainly occur at the module design and computational implementation level; the overall architecture has not fundamentally changed.

4 Working Mechanisms of LLMs

Research on LLMs can be conducted from three perspectives: machine learning methods and theory, external prompt experimental analysis, and internal working mechanism research. If we compare LLMs to the human brain, research on working mechanisms corresponds to neuroscience experiments.

4.1 Feature Superposition

The phenomenon of "feature superposition" may exist in each layer of neural networks. The traditional view is that one neuron represents one feature. However, extensive experiments show that this idealized situation is relatively rare in actual networks. On the contrary, the correspondence between neurons and features is often many-to-many: that is, one neuron participates in representing multiple features, and one feature is jointly represented by multiple neurons.

The Anthropic research team proposed the Superposition Hypothesis [2]. Its core idea is: **Through feature superposition, one layer of neurons in a neural network can approximately represent far more features than the number of neurons**, at the cost of some degree of interference between features.

One layer of a neural network (called the actual layer) can be expressed as:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (10)$$

where $\mathbf{x}$ is the input vector located in the input space; $\mathbf{W}$ is the weight matrix; $\mathbf{b}$ is the bias vector; $\mathbf{h}$ is the output vector or feature vector; ReLU is the activation function.

Feature superposition theory indicates that there exists a wider hypothetical neural network layer that uses more neurons to explicitly represent a large number of features:

$$\bar{\mathbf{h}} = \text{ReLU}(\bar{\mathbf{W}}\mathbf{x} + \bar{\mathbf{b}}) \quad (11)$$

where $\bar{\mathbf{h}}$ is the feature vector, each dimension of which corresponds to a feature, and satisfies $\dim(\bar{\mathbf{h}}) \gg \dim(\mathbf{h})$, that is, the dimension of the wide layer is much larger than that of the actual layer; $\bar{\mathbf{W}}$ and $\bar{\mathbf{b}}$ are the weight matrix and bias vector.

Importantly, the feature vector $\bar{\mathbf{h}}$ of the wide layer is sparse, while the feature vector $\mathbf{h}$ of the actual layer is dense. Sparsity means that for each input, only a few features are activated (for example, only a few dozen out of ten thousand features are activated), making interference between different features small.

The feature superposition hypothesis holds that there is an approximate equivalence relationship between the wide layer and the actual layer. Specifically, both can approximately reconstruct the input vector $\mathbf{x}$ through linear transformation to obtain approximate reconstruction $\hat{\mathbf{x}}$, as shown in Figure 12. Therefore, it can be considered that **in the actual network, the sparse feature vector $\bar{\mathbf{h}}$ is compressed into the dense feature vector $\mathbf{h}$** . The

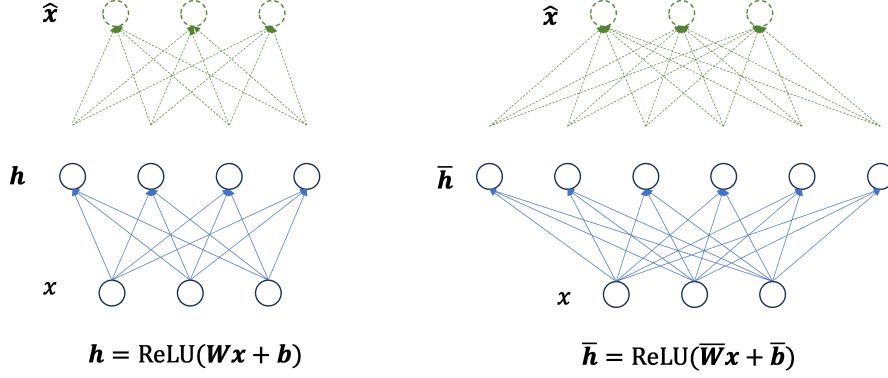


Figure 12: Original feedforward neural network and approximately equivalent wider neural network

sparse feature vector $\bar{\mathbf{h}}$ is nearly independent of each other (non-superposed), while the dense feature vector $\mathbf{h}$ is superposed.

High-dimensional geometric theory provides certain guarantees for the existence of wide layers. Under the conditions of relevant definitions, the number of nearly orthogonal basis vectors in n -dimensional space can reach exponential order of n . Assuming $\bar{\mathbf{v}}_1, \bar{\mathbf{v}}_2, \dots, \bar{\mathbf{v}}_m$ is a set of nearly orthogonal basis vectors in the input space, the input vector can be approximately decomposed as:

$$\mathbf{x} \approx \sum_{i=1}^m \bar{h}_i \bar{\mathbf{v}}_i, \quad \bar{\mathbf{h}} = (\bar{h}_1, \bar{h}_2, \dots, \bar{h}_m) \quad (12)$$

Here each basis vector $\bar{\mathbf{v}}_i$ and its activation value $\bar{h}_i$ correspond to a feature. The basis vectors are nearly orthogonal and the feature vector is sparse, which can well represent the input vector and reduce interference between features.

The feature superposition hypothesis holds that deep learning methods can achieve this effect, that is, the actual layer (dense feature vector) achieves compression of the wide layer (sparse feature vector), or the actual layer (dense feature vector) implicitly contains the wide layer (sparse feature vector). There are two main reasons for this.

First, the number of features contained in a general input vector $\mathbf{x}$ is sparse. For example, when processing the token "bridge" in "I visited the Golden Gate Bridge," in the wide layer feature vector $\bar{\mathbf{h}}$, only a few features may be activated (such as "Golden Gate Bridge," "San Francisco," "bridge structure," "tourist attraction," etc.), while other features all have zero values. This sparsity ensures that interference between a large number of nearly orthogonal features is small enough, making the superposition mechanism practically feasible.

Second, during the training process, neural networks minimize loss functions through gradient descent. When networks face the two goals of "representing as many features as possible" and "using as few neurons as possible," feature

superposition becomes a natural optimization result. In addition, the use of ReLU activation functions also promotes sparsification of feature vectors, because weaker activation values are truncated to zero.

The rationality of the above feature superposition hypothesis has been verified in Anthropic’s toy model simulation experiments and has received further support in subsequent development and application of Sparse Autoencoders.

4.2 SAE: Feature Analysis

Sparse Autoencoders (SAE) can be used to analyze neural networks and discover interpretable features in them. In LLM interpretability research, they are typically applied to the residual stream of Transformers, that is, on the output representation vectors of each layer.

SAE forms a complementary relationship with feature superposition theory. Feature superposition can be viewed as a compression process: the model implicitly represents input vectors through high-dimensional and sparse feature vectors. SAE can be viewed as a "decompression" method: decomposing input vectors into high-dimensional and sparse feature vectors. This "compression-decompression" relationship makes SAE an important tool for studying and analyzing feature superposition phenomena.

SAE consists of an encoder and a decoder. First, the encoder converts input vectors into high-dimensional and sparse feature vectors through nonlinear transformation:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e) \quad (13)$$

where $\mathbf{x} \in \mathbb{R}^n$ is the residual stream from a certain layer of LLM, $\mathbf{W}_e$ is the encoder weight matrix, $\mathbf{b}_e$ is the bias vector, $\mathbf{h} \in \mathbb{R}^m$ is the feature vector, satisfying $m \gg n$, that is, the feature dimension is much larger than the input dimension.

The decoder reconstructs the original input vector from the feature vector through linear transformation:

$$\hat{\mathbf{x}} = \mathbf{W}_d \mathbf{h} + \mathbf{b}_d \quad (14)$$

where $\mathbf{W}_d$ is the decoder weight matrix and $\mathbf{b}_d$ is the bias vector. The decoder uses linear transformation (without activation function), which is consistent with the assumption of "linear combination of features" in feature superposition theory.

SAE training trades off between two objectives: on the one hand, making the reconstructed vector $\hat{\mathbf{x}}$ as close as possible to the original input vector $\mathbf{x}$; on the other hand, introducing L_1 or approximate L_0 regularization to promote sparsity of the feature vector $\mathbf{h}$.

In practice, SAE training requires extracting large amounts of data from the target LLM. Input large-scale corpora into LLM and collect activation vectors (such as residual streams of each layer) generated by the model when processing each token. These vectors $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ constitute the SAE training dataset.

After training is completed, for any input $\mathbf{x}$, the encoder output $\mathbf{h}$ usually exhibits strong sparsity.

Sparse activation features obtained through SAE are consistent with predictions of feature superposition theory, that is, models may encode far more potential concepts than the number of neurons. For example, researchers have successfully extracted hundreds of thousands to millions of features when analyzing large language models, some of which have obvious semantic meanings, such as features related to entities (such as "Golden Gate Bridge") or behaviors (such as "sycophancy").

Analysis shows that features in large language models often exhibit certain hierarchical structures: shallow layers are basically features representing input lexical and simple grammatical features; middle layers have a large number of complex grammatical and basic semantic features; deep layers are mainly complex semantic, reasoning implementation, and output expression features.

4.3 Memory Mechanism

ByteDance’s work proposed the Function Token Hypothesis, revealing the basic characteristics of LLM memory mechanisms [8]. The Function Token Hypothesis holds that feature memory in LLMs revolves around function tokens, and feature retrieval in a context is also conducted through function tokens.

Function tokens refer to the most frequently occurring tokens in training corpora, most of which correspond to function words in linguistics and play key roles in grammar and context connection. For example, the article "the," punctuation marks (commas, periods), line breaks, etc. In contrast are content tokens, which express clear and rich semantic information. Statistics show that in large-scale pre-training corpora, the top 100 or so high-frequency tokens account for about 40% of all token occurrences.

In the LLM pre-training stage, the learning process exhibits significant characteristics centered on function tokens. By decomposing and observing training loss according to the four combinations of function tokens and content tokens, results show that the loss function of "function token $\rightarrow$ content token" decreases most slowly. That is, predicting the next content token based on function tokens is the most difficult. From a linguistic perspective, this is reasonable because function tokens often mark the end of the previous language unit (chunk), and predicting content tokens after it requires accurate understanding of the entire context from the beginning to the current position. It can be inferred that it is precisely this most difficult prediction task that becomes the dominant force driving model optimization.

Another finding is that function tokens can activate most features during training (activating different sparse features in different contexts). A bipartite graph is established between function tokens and features. If a function token activates a certain feature in a certain context, an edge is established between them. As training deepens, edges on the bipartite graph continue to increase. Finally, a small number of function tokens can establish connections with most features; the top 10 high-frequency tokens activate 70% of features, that is,

these function tokens can (in different contexts) activate most features. There is also a power-law distribution here.

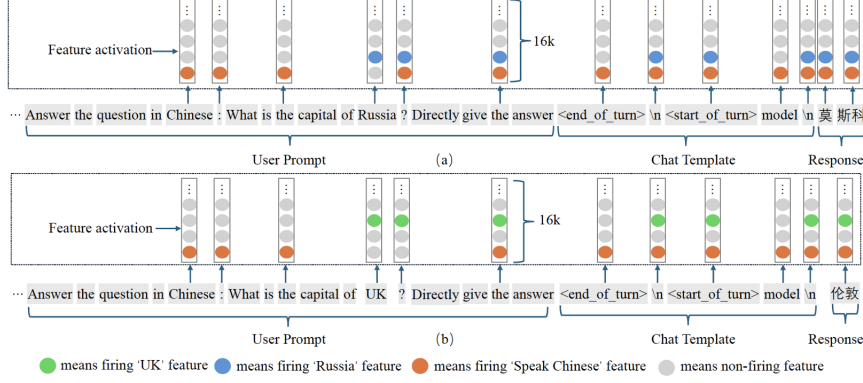


Figure 13: Function tokens play a core role in memory retrieval during LLM reasoning.

During the reasoning process, function tokens play a core role in memory retrieval. They can dynamically activate the most predictive features from the context, thereby guiding the generation of the next token. For example, as shown in Figure 13, when the prompt is "Answer the question in Chinese: What is the capital of Russia?", function tokens (such as colon ":" and line breaks) activate features such as "answer in Chinese" and "Russia" in the context, while suppressing irrelevant features, ultimately guiding the model to generate the answer "Moscow" in Chinese. This dynamic feature selection and combination ability is precisely the key characteristic that distinguishes function tokens from content tokens.

The reason why function tokens play such a key role in LLMs is the result of the combined effects of training objectives, learning algorithms, model architecture, and language characteristics. First, the training objective of next token prediction (cross-entropy loss) requires the model to maximize prediction accuracy, and gradient descent algorithms always prioritize reducing the parts with the largest loss. Second, the feedforward network layer in the Transformer architecture can well represent and memorize knowledge (features), and the self-attention layer can effectively combine lower-order knowledge (features) into higher-order knowledge (features). Finally, the structural characteristics of natural language itself play a decisive role; text is always divided by function tokens into nested chunks (which can be phrases, sentences, or paragraphs). Therefore, predicting after function tokens requires understanding the entire context semantics from the beginning of the text to that position. This is an extremely challenging task that prompts function tokens to acquire the ability to connect most features during training and reactivate the most predictive features during reasoning.

The Function Token Hypothesis has profound implications for LLM training practice. The most important point is that the "format" of training data is crucial. Results from multiple studies confirm this. In the post-training stage, only a small number of training steps are needed to significantly improve the model's instruction following, chain-of-thought reasoning, and other capabilities. This may be because post-training activates features learned during pre-training by adjusting the activation patterns of function tokens. For example, function tokens (such as "thus") can significantly improve reasoning performance in reinforcement learning training.

4.4 CLT: Circuit Analysis

Circuits refer to computational graphs connecting features across layers in LLMs, used to represent how features in the model are activated and propagated. Because SAE can only see features in a single layer, it has significant limitations. To analyze cross-layer feature connections and influence relationships, researchers proposed the CLT (Cross Layer Transcoder) method.

The working principle of CLT is: taking the residual stream $\mathbf{x}^{(l)}$ of a certain layer as input, the model maps it to the residual streams of subsequent layers $\mathbf{x}^{(l+1)}, \mathbf{x}^{(l+2)}, \dots, \mathbf{x}^{(L)}$. In this way, CLT can learn a cross-layer aligned feature dictionary, capturing feature influence relationships between different layers.

CLT has a feature extraction module similar to SAE at each layer, but its optimization objective is very different. The output of each layer is the reconstructed residual streams of subsequent layers. It consists of nonlinear transformation (corresponding to encoder), linear transformation (corresponding to cross-layer mapping), and linear decoding transformation:

$$\mathbf{h}^{(l)} = \text{ReLU}(\mathbf{W}_e^{(l)} \mathbf{x}^{(l)} + \mathbf{b}_e^{(l)}) \quad (15)$$

$$\mathbf{h}^{(l+k)} = \mathbf{T}^{(l \rightarrow l+k)} \mathbf{h}^{(l)} + \mathbf{b}_T^{(l+k)} \quad (16)$$

$$\hat{\mathbf{x}}^{(l+k)} = \mathbf{W}_d^{(l+k)} \mathbf{h}^{(l+k)} + \mathbf{b}_d^{(l+k)} \quad (17)$$

where $\mathbf{W}_e^{(l)}$ is the encoder weight matrix for layer l , $\mathbf{b}_e^{(l)}$ is the bias vector; $\mathbf{W}_d^{(l+k)}$ is the decoder weight matrix for layer $l+k$, $\mathbf{b}_d^{(l+k)}$ is the bias vector; $\mathbf{T}^{(l \rightarrow l+k)}$ is the cross-layer linear transformation weight matrix that maps features from layer l to layer $l+k$, $\mathbf{b}_T^{(l+k)}$ is the bias vector. Through cross-layer linear transformation, $\mathbf{T}^{(l \rightarrow l+k)}$ achieves linear projection from layer l feature space to layer $l+k$ feature space. The training objective of CLT consists of minimizing reconstruction errors of all layers and sparsity regularization.

Based on CLT analysis results, attribution graphs can be constructed to intuitively display linear mapping relationships of features between model layers, helping understand LLM internal knowledge representation and computational mechanisms.

The construction of attribution graphs is based on trained CLT models. Given a specific input prompt, first run the original Transformer model and record residual streams $\mathbf{x}^{(l)}$ of each layer. Then, use CLT to extract sparse

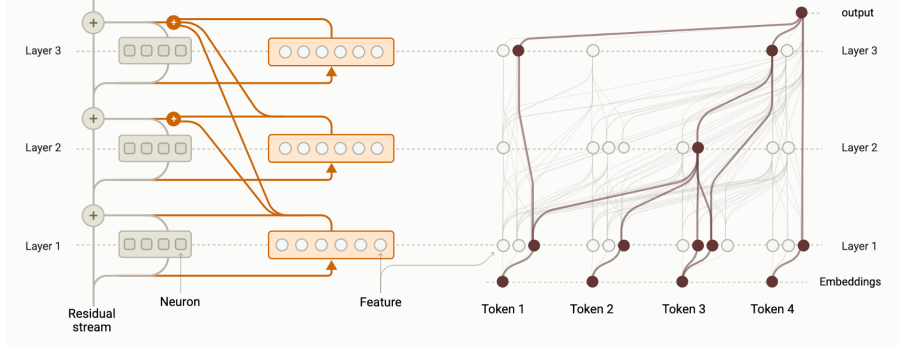


Figure 14: Attribution graph constructed based on CLT for analyzing LLM internal computational mechanisms.

features of each layer $\mathbf{h}^{(l)} = \text{ReLU}(\mathbf{W}_e^{(l)} \mathbf{x}^{(l)} + \mathbf{b}_e^{(l)})$, and analyze mapping relationships between features through cross-layer mapping matrix $\mathbf{T}^{(l \rightarrow l+k)}$.

An attribution graph is a directed acyclic graph representing the computational path from input tokens to output tokens. In this graph, nodes represent activated features or token embeddings in the model, while edges represent influence relationships between nodes. Specifically, if feature i at layer l is activated ($h_i^{(l)} > 0$) and makes significant contributions to the activation of feature j at layer $l + k$ through cross-layer mapping $\mathbf{T}^{(l \rightarrow l+k)}$, then a directed edge from node (i, l) to $(j, l + k)$ is added to the graph.

To improve the interpretability of attribution graphs, pruning techniques are used to simplify the graph. By setting thresholds, only nodes and edges with significant activation strength and contribution are retained. Gradient back-propagation is further used to identify paths that contribute most to the final output. The resulting simplified attribution graph can more clearly display feature activation and reasoning paths in the model, revealing core feature circuits of the model in specific scenarios.

5 Language Understanding and Reasoning of LLMs

5.1 Capabilities of LLMs

5.1.1 Higher-Order Patterns

From their behavioral performance, large language models have demonstrated language and reasoning capabilities equal to or exceeding those of humans. Using the Turing test as a criterion, that is, examining whether they cannot be distinguished from humans in dialogue, LLMs have reached human level.

What LLMs learn is not limited to lower-order patterns of language, but also encompasses higher-order patterns of language and reasoning. This can be intuitively verified in our daily use of LLMs. For example, LLMs can un-

derstand and execute instructions such as "How high is Mount Everest, answer in English," demonstrating their pragmatic capabilities; they can also analyze questions involving conceptual similarities and differences such as "the relationship between Golden Gate Bridge and Golden Arches," showing semantic understanding and integration of world knowledge.

From the perspective of internal mechanism analysis, the existence of conceptual features such as "Golden Gate Bridge" and "sycophancy" in models also reveals that LLMs have semantic and pragmatic understanding capabilities.

Chomsky once criticized LLMs for only learning surface statistical regularities of language. However, the above facts show that this judgment does not hold. However, this does not mean that LLMs are completely equivalent to human language capabilities. In fact, LLM language mechanisms differ significantly from the human brain. For example, human language understanding relies on the collaborative work of two brain regions, Broca's area and Wernicke's area: the former mainly handles grammar, while the latter handles vocabulary.

5.1.2 Overall Mechanism

The overall working mechanism of Large Language Models (LLMs) can be understood from training methods, strategies, algorithms, and models (see Figure 15). Their training method usually includes two stages: pre-training and post-training, and through two-step training fusion, enables the model to simultaneously possess language understanding, generation, and reasoning capabilities in the same system. In terms of mechanism, LLMs are based on autoregressive prediction, that is, gradually predicting the next word based on existing context, and this process can also be seen as a sequential decision-making process.

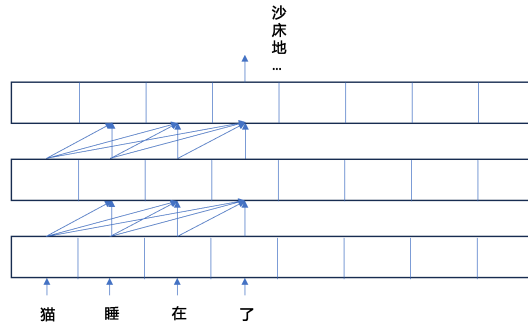


Figure 15: The mechanism of LLMs can be understood from training methods, strategies, algorithms, and models.

In the pre-training stage, the model learns statistical regularities through large-scale corpora; given sufficiently long preceding text, the probability distribution of the next token tends to be more concentrated. In the post-training stage, through human feedback or policy optimization, the model is made more

inclined to generate the most reasonable and human-preferred following text under the same context.

From the perspective of technical implementation, learning strategies are responsible for defining prediction objectives and optimization directions, algorithms are used to adjust model parameters to achieve optimal objectives, and model structure determines expressive power, achieving feature composition through self-attention mechanisms, feature detection (nonlinear transformation) through feedforward networks, and hierarchical representation through multi-layer structures. These all play important roles in LLM’s human-like language and reasoning capabilities.

Model performance improvement also exhibits obvious scale effects: as data volume, parameter scale, and computational resources increase, model capabilities can produce qualitative leaps. At the same time, data quality and training design (such as high-quality corpora and system prompt design) are also crucial to effectiveness.

LLMs trained in this way form a large number of features representing various different concepts. According to different contexts, these different features are activated and dynamically form circuits, achieving complex language processing and reasoning mechanisms.

5.2 Comparison with Human Capabilities

Table 2 compares the capabilities of LLMs and humans. It can be seen that LLMs already possess capabilities comparable to or even exceeding human levels in language and reasoning tasks. However, in other capability dimensions, not only may the mechanisms fundamentally differ, but their performance is not simply comparable. The following provides a brief explanation and discussion of this.

Table 2: Comparison of LLM and human capabilities

Capability Dimension	LLM
Language Capability	Reaches or exceeds human level
Reasoning Capability	Reaches or exceeds human level
Factual Knowledge	Has hallucinations
Multimodal Processing	Does not inherently possess, but can be extended to
Mathematical Ability (computation, logical reasoning)	Has certain abilities but with limitations
Thinking Ability	Does not possess human-like mechanisms combining
Emotion	Has emotional elements in language and reasoning,
Creativity	Has a certain degree of creativity
Consciousness	Different mechanism

Hallucinations essentially stem from errors in judging facts. LLMs themselves cannot solve the hallucination problem. Because they learn statistical regularities in language data. Theory proves that under certain assumptions, hallucinations will inevitably occur with certain probability during language

generation [3]. The hallucination problem can be mitigated through other mechanisms, such as Retrieval-Augmented Generation (RAG).

Human thinking involves multiple aspects, not only language, reasoning, and mathematics, but also closely related to the five senses (vision, hearing, touch, smell, taste) and the motor system. The embodied cognition hypothesis believes that the thinking process is not pure symbolic computation, but simulation in the mind based on sensorimotor experience of the body. Current Multimodal Large Language Models (MLLMs) can process language reasoning in association with visual, auditory, and other information, but their reasoning process usually occurs in the language representation space. This processing method has fundamental differences from human thinking based on embodied experience, which is rich and conscious. Recently, some work has attempted multimodal reasoning, allowing models to "see and think" simultaneously; but these methods are still relatively preliminary explorations. Therefore, from the perspective of embodied cognition, MLLM thinking capabilities still have significant gaps compared to humans.

LLMs neither reason based on formal logical rules (such as propositional logic) nor perform arithmetic operations according to computational rules. They can simulate through their generation mechanisms and exhibit certain heuristic reasoning and computational capabilities, but when handling complex problems, they are prone to errors due to lack of rigor. Therefore, LLMs have limitations in this regard.

Whether LLMs have creativity is still an open question, which also depends on the definition of creativity. Innovation is divided into incremental innovation and disruptive innovation. Through observation, it can be found that LLMs should also possess incremental innovation capabilities. As for disruptive innovation, such as the establishment of relativity theory, whether LLMs can achieve this is still uncertain. Ilya Sutskever believes that LLMs can do interpolation, but extrapolation is an open question, which is a similar idea.

LLMs do not have consciousness mechanisms corresponding to humans, although we may feel like interacting with real people when conversing with LLMs. Consciousness refers to the awareness of internal body and external environment felt in the human mind. Consciousness is a state the human brain is in when awake; other states relative to it are deep sleep, coma, and death. Consciousness is subjective, mental activity centered on the self for each person; for each of us, our own consciousness is continuous, consistent, and stable. At the same time, consciousness is also objective; it corresponds to high-level processing of the human brain's nervous system, supported by low-level processing of the brain's nervous system. Global Workspace Theory believes that consciousness is the global broadcast of information in the brain.

6 Conclusion

This article discusses LLM's language and thinking capabilities from the perspectives of implementation methods and working mechanisms. We believe that

in a certain sense, LLMs have reached or exceeded human language and thinking levels.

Large Language Models (LLMs) essentially learn patterns in language data, that is, regularities, and can apply these patterns, especially higher-order patterns, to exhibit intelligent behavior.

The capabilities learned by LLMs can be simply summarized as Next Token Prediction, but in fact are the result of highly effective implementation principles and methods; data scale and model scale also play important roles. Pre-training through data compression enables models to find regularities in large amounts of diverse data; post-training through policy models fine-tunes models to more reasonably complete tasks and implement reasoning. Gradient descent combined with backpropagation allows model parameters to be adjusted to better represent data patterns. Transformer architecture can well represent hierarchical, diverse, and flexibly combined paradigms in data through attention, FFN, and residual mechanisms.

Complex working mechanisms exist inside LLMs. Analysis tools such as SAE and CLT have discovered that there really are large numbers of features at different levels in models, and these features form circuits that can be memorized during learning and retrieved during prediction.

Large language models actually represent a general technology and paradigm for learning patterns in sequential data, which can be used not only for language data but also for other types of data such as images.

References

- [1] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.
- [2] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition, 2022.
- [3] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate, 2025.
- [4] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [5] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for

deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016.

- [6] Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. Reft: Reasoning with reinforced fine-tuning, 2024.
- [7] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization, 2017.
- [8] Shaohua Zhang, Yuan Lin, and Hang Li. Memory retrieval and consolidation in large language models through function tokens, 2025.
- [9] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, and Junyang Lin. Stabilizing reinforcement learning with llms: Formulation and practices, 2025.